

Evaluation Matrices in NLP

- Accuracy
- Recall
- Precision
- F-measure
- Confusion Matrix
- ROUGE Score
- BLEU Score
- METEOR
- MRR
- Cohen Kappa

Confusion Matrix

A **confusion matrix** is a **performance measurement tool** used for classification tasks.

It shows how many predictions were:

- Correct (True)
- Incorrect (False)

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Metric	What it Tells You
TP	Model correctly predicted positive cases
FP	Model falsely predicted positive for negatives
FN	Model missed actual positives
TN	Model correctly identified negatives

Example

- Imagine a **sentiment analysis** system tested on 10 texts:

Text #	Actual Sentiment	Predicted Sentiment
1	Positive	Positive
2	Positive	Positive
3	Positive	Negative ✖
4	Negative	Negative
5	Negative	Positive ✖
6	Negative	Negative
7	Positive	Positive
8	Negative	Negative
9	Positive	Negative ✖
10	Negative	Negative

Let's count:

- TP (correct Positive) = 3 (Text 1, 2, 7)
- FN (missed Positive) = 2 (Text 3, 9)
- FP (wrongly predicted Positive) = 1 (Text 5)
- TN (correct Negative) = 4 (Text 4, 6, 8, 10)

Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive (5)	TP = 3	FN = 2
Actual Negative (5)	FP = 1	TN = 4

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{Total} = \frac{3 + 4}{10} = 0.7$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{3}{3 + 1} = 0.75$$

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{3}{3 + 2} = 0.6$$

F1 Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} = 2 \times \frac{0.75 \times 0.6}{0.75 + 0.6} \approx 0.666$$

Accuracy

- **Accuracy** measures the **proportion of total predictions** that were **correct**.

(“Out of all the predictions the model made, how many were right?”)

Accuracy Formula

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Example

Now, suppose you tested your model on **10 emails**, and you have the following:

Email #	Actual Label	Predicted Label
1	Spam (1)	Spam (1)
2	Not Spam (0)	Not Spam (0)
3	Spam (1)	Not Spam (0)
4	Spam (1)	Spam (1)
5	Not Spam (0)	Spam (1)
6	Not Spam (0)	Not Spam (0)
7	Spam (1)	Spam (1)
8	Not Spam (0)	Not Spam (0)
9	Spam (1)	Not Spam (0)
10	Not Spam (0)	Not Spam (0)

Email #	Actual Label	Predicted Label	Result
1	Spam (1)	Spam (1)	True Positive (TP)
2	Not Spam (0)	Not Spam (0)	True Negative (TN)
3	Spam (1)	Not Spam (0)	False Negative (FN)
4	Spam (1)	Spam (1)	TP
5	Not Spam (0)	Spam (1)	False Positive (FP)
6	Not Spam (0)	Not Spam (0)	TN
7	Spam (1)	Spam (1)	TP
8	Not Spam (0)	Not Spam (0)	TN
9	Spam (1)	Not Spam (0)	FN
10	Not Spam (0)	Not Spam (0)	✅ TN

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{3 + 4}{3 + 4 + 1 + 2} = \frac{7}{10} = 0.7$$

Final Accuracy is 70%

Interpretation

- Your model got **7 out of 10 emails correct**.
- It correctly identified 3 spam and 4 not-spam emails.
- It **missed 2 spam emails** (FN), which could be critical depending on your application.
- It **flagged 1 legitimate email as spam** (FP), which could annoy users.

Advantages:

- Easy to understand and calculate
- Works well when **classes are balanced**
- A good **first check** for classification tasks

When Accuracy Can Be Misleading when class are imbalanced. (In imbalanced datasets, one class appears **much more frequently** than others) A model can achieve **high accuracy** just by predicting the majority class — but **still perform poorly**

Example

Suppose we have **1000 emails**:

- **950 are Not Spam** (label: 0)
- **50 are Spam** (label: 1)

Your model is **naive** and always predicts "Not Spam" for every email.

Model Predictions

Class	Actual Count	Predicted Count
Not Spam (0)	950	950 (TN)
Spam (1)	50	0 (FN)

$$\text{Accuracy} = \frac{TP + TN}{Total} = \frac{0 + 950}{1000} = 0.95$$

Despite a **95% accuracy**, the model **completely failed** to detect any spam emails.
This is **misleading**

Recall & Precision

Recall measures a model's ability to find **all the actual positive cases** in your data. (“Out of all the things that were actually *positive*, how many did the model *successfully find*?”)

Precision measures out of everything the model predicted as *positive*, how many were actually *correct*?”

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Precision** is about **quality**
- **Recall** is about **coverage**

Given:

- **Total emails = 100**
- **Actual spam = 20**
- **Actual not spam = 80**
- **Model predicted 15 as spam, and all 15 were actual spam**
- **Missed 5 spam emails, meaning they were predicted as not spam**

Calculate Accuracy, Precision and Recall

Metric	Meaning	Value
TP	True Positives (Spam predicted as Spam)	15
FN	False Negatives (Spam predicted as Not Spam)	5
FP	False Positives (Not Spam predicted as Spam)	0
TN	True Negatives (Not Spam predicted as Not Spam)	80

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{Total} = \frac{15 + 80}{100} = \frac{95}{100} = \boxed{0.95 \text{ or } 95\%}$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{15}{15 + 0} = \frac{15}{15} = \boxed{1.0 \text{ or } 100\%}$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{15}{15 + 5} = \frac{15}{20} = \boxed{0.75 \text{ or } 75\%}$$

F-measure

- The **F1 Score** is the **harmonic mean** of **Precision** and **Recall**:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

It balances both:

- High **precision** (few false positives)
- High **recall** (few false negatives)

example:

- **Precision** = 1.0 (or 100%)
- **Recall** = 0.75 (or 75%)

$$\text{F1 Score} = 2 \times \frac{1.0 \times 0.75}{1.0 + 0.75} = 2 \times \frac{0.75}{1.75} = \frac{1.5}{1.75} \approx 0.857$$

Interpretation

- The **F1 Score** is a **balanced metric**:
 - Even though precision was perfect, the recall (75%) pulls the F1 score slightly down.
- It's especially useful when:
 - The dataset is **imbalanced**

ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

It measures the **overlap between the generated text and a reference (human-written) text** using **n-grams, word sequences, and word pairs**.

ROUGE is used to **evaluate the quality of generated language** by comparing it to one or more reference texts. Common tasks include:

- Text summarization**
- Machine translation**
- Dialogue systems**
- Paraphrase generation**

Types of ROUGE Scores

ROUGE Type

What It Measures

ROUGE-N

Overlap of n-grams (e.g., ROUGE-1 = unigrams, ROUGE-2 = bigrams)

ROUGE-L

Longest Common Subsequence (LCS)

ROUGE-W

Weighted LCS

ROUGE-S

Skip-bigram co-occurrence (with gaps allowed)

ROUGE-N is most Common type. **ROUGE-1: Unigram overlap, ROUGE-2: Bigram overlap**

Formula (Recall-Oriented):

$$\text{ROUGE-N} = \frac{\text{Number of overlapping n-grams}}{\text{Number of n-grams in reference}}$$

Example

Reference Summary:

"The cat sat on the mat."

Generated Summary:

"The cat sat on a mat."

ROUGE-1 (Unigrams):

- Reference unigrams: ["the", "cat", "sat", "on", "the", "mat"]
- Generated unigrams: ["the", "cat", "sat", "on", "a", "mat"]
- Overlap: ["the", "cat", "sat", "on", "mat"] → 5 overlaps
- ROUGE-1 Recall:

$$\frac{5}{6} \approx 0.83 \text{ or } 83\%$$

ROUGE-2 (Bigrams):

- Reference bigrams: ["the cat", "cat sat", "sat on", "on the", "the mat"]
- Generated bigrams: ["the cat", "cat sat", "sat on", "on a", "a mat"]
- Overlap: ["the cat", "cat sat", "sat on"] → 3 overlaps
- ROUGE-2 Recall:

$$\frac{3}{5} = 0.60 \text{ or } 60\%$$

BLEU Score

BLEU stands for:

Bilingual **E**valuation **U**nderstudy

It evaluates how **close** a machine-generated sentence is to a **human-written reference** by comparing **n-grams** (i.e., contiguous word sequences).

BLEU is:

- **Precision-oriented** (not recall)
- Based on **n-gram overlap**
- Measures **how many n-grams in the candidate appear in the reference**
- Ranges from **0 to 1** (or 0% to 100%)

BLEU Score Components

- **1. Modified n-gram Precision**
- Counts overlapping n-grams between the generated text and reference, but **clips** counts to avoid inflating the score.
- **2. Brevity Penalty (BP)**
- Penalizes **shorter generated texts**, since they can have artificially high precision.

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

/here:

- p_n = modified precision for n-grams
- w_n = weight for each n-gram (commonly equal weights, e.g., 0.25 for 1–4 grams)
- **BP** = brevity penalty

Example

Reference: "The cat is on the mat"

Candidate (Generated): "The cat the cat on the mat"

Step 1: Unigrams (BLEU-1)

- Reference unigrams: ["The", "cat", "is", "on", "the", "mat"]
- Candidate unigrams: ["The", "cat", "the", "cat", "on", "the", "mat"]

Count matching unigrams (clip to max reference count):

Word	Candidate Count	Reference Count	Clipped Count
the	3	2	2
cat	2	1	1
on	1	1	1
mat	1	1	1
is	0	1	0

Total matches (clipped) = 5

Total candidate unigrams = 7

$$\text{BLEU-1} = \frac{5}{7} \approx 0.714$$

- **Step 2: Brevity Penalty (BP)**
- Reference length = 6
- Candidate length = 7
- Since candidate > reference → **no penalty**

$$BrevityPenalty = \min\left(1, \frac{Machine\ Translation\ Output\ Length}{Maximum\ Reference\ Output\ Length}\right)$$

BP=1

Final BLEU-1 Score:

$$BLEU-1 = 1 \times 0.714 = \boxed{0.714 \text{ or } 71.4\%}$$

BLEU Variants

BLEU-n

Description

BLEU-1

Unigram precision

BLEU-2

Bigram precision

BLEU-4

Up to 4-grams (standard default)

Strengths & Limitations

Pros:

- Fast and easy to compute
- Good for **automated evaluation**
- Widely used in **machine translation benchmarks**

Cons:

- Insensitive to meaning or semantics**
- Rewards **surface-level overlap**
- Penalizes **valid paraphrases**

ROUGE and BLEU Comparison

Feature	BLEU	ROUGE
Full Form	Bilingual Evaluation Understudy	Recall-Oriented Understudy for Gisting Evaluation
Focus	Precision (How much generated text matches reference)	Recall (How much reference text is captured in the output)
Main Use Case	Machine Translation	Text Summarization (especially abstractive)
N-gram Based?	Yes (1-gram to 4-grams)	Yes (ROUGE-N, ROUGE-L, ROUGE-S)
Penalizes Short Outputs?	Yes (Brevity Penalty)	No (prefers longer outputs with more recall)
Handles Multiple References?	Yes	Yes
Handles Word Order?	Partially (via n-grams)	Better with ROUGE-L (Longest Common Subsequence)
Common Variants	BLEU-1 to BLEU-4	ROUGE-1, ROUGE-2, ROUGE-L, ROUGE-S
Output Score Range	0 to 1 (or 0% to 100%)	0 to 1 (or 0% to 100%)
Interpretability	Lower values still usable (0.3–0.5 common)	ROUGE often produces higher scores for same outputs

METEOR: Metric for Evaluation of Translation with Explicit ORdering

It was developed to **overcome limitations of BLEU** by considering:

- **Synonyms**
- **Stemming**
- **Word order**
- **Recall** as well as **precision**

Unlike **BLEU**, which only focuses on **n-gram precision**, **METEOR** aims to **align words meaningfully** between the reference and candidate sentence.

- It provides a **more flexible and linguistically aware** evaluation, especially helpful in:
- Handling **paraphrases**
- Rewarding **semantic similarity**, not just exact match

How METEOR Works

- **Align candidate and reference sentences** using:
 - Exact word matches
 - Stemming (e.g., "run" vs. "running")
 - Synonyms (using WordNet or other lexicons)
- **Compute:**
 - **Precision** = fraction of matched unigrams over candidate unigrams
 - **Recall** = fraction of matched unigrams over reference unigrams
- Combine them into an **F-score** (with **Recall weighted higher** than Precision)
- Apply a **penalty** for **fragmentation** (how many matched chunks are out of order)

Step 1: Compute Precision & Recall

- **Precision** = $\frac{\text{Matched Unigrams}}{\text{Total words in candidate}}$
- **Recall** = $\frac{\text{Matched Unigrams}}{\text{Total words in reference}}$

Step 2: Compute F1 Score

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Step 3: Apply Penalty for Fragmentation

$$\text{Penalty} = \gamma \cdot \left(\frac{\text{chunks}}{\text{matches}} \right)^\theta$$

$$\text{METEOR} = F1 \cdot (1 - \text{Penalty})$$

- Default values: $\gamma = 0.5, \theta = 3$

Example

Reference:

"The cat is sitting on the mat"

Candidate:

"A cat sits on the rug"

METEOR will:

- Recognize "cat" = "cat" (exact match)
- "sits" $\approx$ "sitting" (stemming match)
- "rug" $\approx$ "mat" (synonym match via WordNet)

So even with **different words**, METEOR will reward **semantic overlap**.

MRR

- **Mean Reciprocal Rank (MRR)** is a ranking evaluation metric commonly used in **information retrieval**, **question answering**, and **recommendation systems**.
- It measures how highly the **first correct answer** is ranked among a list of results.

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$$

Where:

- N = number of queries or examples
- rank_i = position of the **first correct result** in the list for query i

MRR Characteristics

- **Best score:** 1 (correct answer is ranked 1st every time)
- **Worst score:** Close to 0 (correct answers are ranked very low)
- Only the **first relevant result** matters.

Let's say we have a QA system that returns ranked lists of answers for 3 different questions:

♦ Query 1:

Candidate answers:

1. "London"
2. "Paris"
3. "Berlin" ←  correct answer at rank 3

Query	Correct Answer Rank	Reciprocal Rank (1/rank)
Q1	3	$1/3 \approx 0.333$
Q2	1	$1/1 = 1.0$
Q3	— (not found)	0

♦ Query 2:

Candidate answers:

1. "Einstein" ←  correct answer at rank 1

♦ Query 3:

Candidate answers:

1. "Tesla"
2. "Newton"
3. "Edison"

(No correct answer in top 3) ← 

$$\text{MRR} = \frac{0.333 + 1.0 + 0}{3} = \frac{1.333}{3} \approx 0.444$$

Cohen's Kappa

Cohen's Kappa is used when **two annotators (or systems)** label data independently, and you want to evaluate how much they **agree — beyond what would be expected by chance.**

It measures the agreement between two raters **adjusted for chance.**

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

Where:

- p_o = **Observed agreement** (the proportion of times both raters agreed)
- p_e = **Expected agreement by chance** (based on the distribution of categories from both raters)

Interpretation of κ values

κ Value	Strength of Agreement
< 0	Poor
0.01–0.20	Slight
0.21–0.40	Fair
0.41–0.60	Moderate
0.61–0.80	Substantial
0.81–1.00	Almost perfect

Example

- Suppose two annotators are classifying 100 items into Positive/Negative.

	Rater B Positive	Rater B Negative	Row Total
Rater A Positive	40	10	50
Rater A Negative	5	45	50
Column Total	45	55	100

Step 1: p_o (Observed agreement)

$$p_o = \frac{40 + 45}{100} = 0.85$$

Step 2: p_e (Expected agreement)

- For Positive: $(50 \times 45)/100 = 22.5$
- For Negative: $(50 \times 55)/100 = 27.5$

$$p_e = \frac{22.5 + 27.5}{100} = 0.50$$

Step 3: Plug into formula

$$\kappa = \frac{0.85 - 0.50}{1 - 0.50} = \frac{0.35}{0.50} = 0.70$$

Cohen's Kappa = 0.70 → Substantial Agreement

When is Cohen's Kappa Needed?

1.Data Labeling/Annotation Tasks

1. Comparing annotations from two human annotators on sentiment, intent, or named entity labels.

2.Model vs. Human Evaluation

1. Comparing a model's predictions to human-annotated ground truth.

Summary

Task Type	Metric	Used For
Classification	Accuracy, F1	Sentiment, Spam, Intent classification
Sequence Labeling	Precision, Recall	POS tagging, NER
MT / Summarization	BLEU, ROUGE, METEOR	Translation, Summarization
QA / Retrieval	MRR	Question answering, search
Text Gen Quality	BERTScore	Fluency and semantic evaluation